Indian Institute of Computer and Technology

A PROJECT REPORT ON

AI-Powered Fake News Detection Using Text Classification

Submitted in partial fulfilment of the requirements for
Bachelor of Computer Applications (BCA) — Semester IV

Submitted by:
Jahnvi Pandey

Reg. No. : 605971

Under the Guidance of:
Dr. Ashok Gopalakrishnan

2026

CERTIFICATE

This is to certify that the project report entitled “AI-Powered Fake News Detection Using Text Classification” has been carried out by Jahnvi Pandey (605971), an intern of Indian Institute of Computer and Technology, under my supervision and guidance in partial fulfilment of the requirements for the award of the degree.

The work presented in this report is based on the student’s own implementation and has been found satisfactory for submission, to the best of my knowledge.

Date: ____28/07/2026__
Place: _____

(Signature of Guide)
Dr. Ashok Gopalakrishnan

ACKNOWLEDGEMENT

I want to express my sincere gratitude to my project guide for the guidance and encouragement provided throughout the course of this project. Their feedback at each stage helped me approach the problem in a more structured manner and correct several implementation issues along the way.

I am also thankful to the faculty of the Indian Institute of Computer and Technology or providing the resources and lab facilities required to carry out this work, and to my classmates for the useful discussions during the development of the preprocessing pipeline.

Finally, I would like to thank my family for their constant support and patience during the preparation of this project.

Jahnvi Pandey

ABSTRACT

The rapid growth of digital news platforms and social media has made it considerably easier for unverified and misleading information to circulate alongside genuine reporting. Manually verifying every article that appears online is not practical at the scale modern platforms operate at, which motivates the use of automated, text-based classification methods to flag content that may be fake. This project works toward such a system for the binary classification of news articles as Real or Fake using Natural Language Processing techniques.

The work reported here covers the data acquisition and preprocessing stage of the pipeline. A structured dataset of 1,200 labelled news records, each carrying fields such as full article text, source name, publication category, country of origin, and several auxiliary indicators (sentiment score, clickbait score, emotional word ratio, readability score and source trust score), was loaded and examined for structural and quality issues. No missing values or duplicate rows were found in the dataset. A text-cleaning function was implemented to normalise the article text through lowercasing, removal of URLs and HTML artefacts, stripping of digits and punctuation, whitespace normalisation and stopword removal using the NLTK English stopword corpus. The cleaned text was tokenised and a word-count feature was derived for each article, after which the processed dataset was exported for later use.

At the current stage, feature vectorisation (TF-IDF), the train-test split, and the training and comparison of classification models (K-Nearest Neighbours, Logistic Regression, Random Forest and a Neural Network) have not yet been implemented and are documented in this report as planned extensions rather than completed work. The report is structured to reflect this honestly: it explains what has been built, why each preprocessing decision was made, and what remains to be done before the classification stage can be evaluated.

Keywords: Fake News Detection, Natural Language Processing, Text Preprocessing, Stopword Removal, Text Classification, Data Cleaning, Tokenisation, Machine Learning

Table of Contents

CERTIFICATE ii
ACKNOWLEDGEMENT ii
ABSTRACT iv
Table of Contents v
1 Introduction 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Importance of Fake News Detection 1
1.4 Objectives 1
1.5 Scope of the Project 2
1.6 Project Workflow 2
1.7 Organization of Report 2
2 Literature Review 3
2.1 Traditional Machine Learning Approaches 3
2.2 NLP-Based and Deep Learning Approaches 3
2.3 Auxiliary Signal-Based Approaches 3
2.4 Comparison of Existing Techniques 3
2.5 Research Gap 4
3 Dataset Description 5
3.1 Dataset Source 5
3.2 Dataset Size and Structure 5
3.3 Target Variable and Class Distribution 5
3.4 Category and Country Distribution 6
3.5 Auxiliary Numeric Features 7
3.6 Missing Values and Duplicate Records 7
3.7 Observations 8
4 Methodology 9
4.1 Library Imports 9
4.2 Data Loading 9
4.3 Data Quality Checks 9
4.4 Handling Missing Values 9
4.5 Text Cleaning Function 10
4.6 Tokenisation 10
4.7 Word Count Feature 10
4.8 Saving the Cleaned Dataset 11
4.9 Feature Engineering and Model Pipeline (Planned) 11

5 Planned Model Development (Future Work) 12
5.1 K-Nearest Neighbours (Planned) 12
5.2 Logistic Regression (Planned) 12
5.3 Random Forest (Planned) 12
5.4 Neural Network (Planned) 12
5.5 Summary 13
6 Model Evaluation (Planned) 14
6.1 Evaluation Metrics 14
6.2 Planned Comparison Methodology 14
6.3 Note on Reporting 14
7 Results and Discussion 15
7.1 Outcome of the Preprocessing Stage 15
7.2 Effect on Word Count 15
7.3 Limitations Observed 15
7.4 Practical Usability at This Stage 15
8 Conclusion 16
8.1 Summary of Work Done 16
8.2 Limitations 16
8.3 Future Work 16
References 17
Appendix — Selected Code Snippets 18
A.1 Imports and Dataset Loading 18
A.2 Missing Value Handling 18
A.3 Text Cleaning Function (final version) 18
A.4 Tokenisation and Word Count 18
A.5 Saving the Cleaned Dataset 18

1. Introduction

1.1 Background

News consumption has largely shifted from print and television to websites, aggregators and social media feeds, where an article can be shared thousands of times within minutes of publication. This speed works in favour of readers when the information is accurate, but it also gives fabricated or misleading stories the same reach as verified reporting, often before any fact-checking organisation has had the chance to respond. Because the presentation of a fake article can be visually indistinguishable from a genuine one, readers frequently judge credibility by tone, headline phrasing or source name rather than by the underlying facts.

Natural Language Processing (NLP) and Machine Learning offer a way to approach this problem computationally. Instead of relying on a human reviewer to read every article, a classification model can be trained to recognise textual patterns — word choice, structure, sentiment, sensationalism — that are statistically associated with fake content. This project explores that idea by building the data pipeline needed to eventually support such a classifier.

1.2 Problem Statement

Given a dataset of news articles labelled as Real or Fake, the problem addressed in this project is to prepare the raw textual data in a form suitable for machine learning, and to lay the groundwork for a system that can automatically classify unseen articles by analysing their content rather than their source alone.

1.3 Importance of Fake News Detection

Automated fake news detection is relevant to several stakeholders. Social media platforms use such systems to reduce the spread of misinformation before it goes viral. News aggregators use them to flag content for manual review. Individual readers benefit indirectly through browser extensions and fact-checking tools built on similar techniques. At a broader level, unchecked misinformation has been linked to public health confusion, financial market manipulation and erosion of trust in legitimate journalism, which is why this remains an active area of applied NLP research.

1.4 Objectives

The objectives set out for this project are as follows:

- To collect and examine a labelled dataset of news articles suitable for binary Real/Fake classification.
- To design and implement a text-cleaning pipeline that removes noise (URLs, HTML tags, digits, punctuation and stopwords) from raw article text.
- To tokenise the cleaned text and derive basic statistical features such as word count.

- To prepare the cleaned dataset so that it can be vectorised (TF-IDF / Bag-of-Words) and used to train classification models in a subsequent phase of the project.
- To document the pipeline clearly enough that model training, evaluation and comparison can be added without reworking the preprocessing stage.

1.5 Scope of the Project

The scope of the work completed so far covers data loading, exploratory checks (shape, column types, missing values, duplicates), text cleaning, tokenisation and word-count feature extraction. Vectorisation of the cleaned text, splitting the data into training and testing sets, and the training and evaluation of classification models (K-Nearest Neighbours, Logistic Regression, Random Forest and a Neural Network) are outside the current scope of the implemented notebook and are treated in this report as planned future work rather than completed results, so that the report accurately reflects what has been built.

1.6 Project Workflow

The workflow implemented in the notebook proceeds in the following order: the dataset is loaded from a local CSV file; basic exploratory checks are run to confirm its structure and quality; missing values in the article text column are handled; a cleaning function is applied to normalise the text; the cleaned text is tokenised; a word-count feature is derived from the token list; and the resulting dataframe is exported to a new CSV file. This sequence is illustrated in Figure 4.1 in Chapter 4.

1.7 Organization of Report

Chapter 2 reviews common approaches to fake news detection reported in the literature. Chapter 3 describes the dataset used in this project in detail. Chapter 4 explains the preprocessing methodology actually implemented in the notebook. Chapter 5 outlines the classification models planned for the next phase of the project. Chapter 6 discusses the evaluation metrics that will be used once those models are trained. Chapter 7 discusses the results obtained so far and the limitations observed. Chapter 8 concludes the report and outlines the remaining work.

2. Literature Review

Fake news detection has been approached from several angles in existing work, ranging from simple keyword and source-credibility checks to deep contextual language models. This chapter summarises the broad categories of techniques commonly used, without attributing specific results to any single publication, since no particular paper was used as a reference implementation for this project.

2.1 Traditional Machine Learning Approaches

A large body of prior work treats fake news detection as a standard text classification problem. Articles are first converted into numerical vectors using Bag-of-Words or TF-IDF representations, and then fed into classical classifiers such as Naive Bayes, Logistic Regression, Support Vector Machines, K-Nearest Neighbours and Random Forests. These methods are attractive because they train quickly, require comparatively little data, and are easy to interpret through feature weights or decision boundaries. Their main limitation is that they rely heavily on surface-level word frequency and generally do not capture word order or long-range context within a sentence.

2.2 NLP-Based and Deep Learning Approaches

More recent approaches make use of word embeddings (Word2Vec, GloVe) and sequence models (RNN, LSTM, GRU) that can capture some ordering information in the text, as well as transformer-based language models such as BERT that are fine-tuned on labelled news data. These models tend to outperform classical methods on large, real-world datasets because they capture contextual meaning rather than isolated word frequency, at the cost of significantly higher computational requirements and the need for larger volumes of training data.

2.3 Auxiliary Signal-Based Approaches

A separate line of work supplements the article text with auxiliary signals such as source credibility, publication metadata, social engagement patterns and sentiment or clickbait scoring, on the reasoning that fake content is not identified by wording alone. The dataset used in this project follows a similar design, since it provides sentiment score, clickbait score, emotional word ratio, readability score and source trust score alongside the article text, which suggests these fields were intended to support this kind of combined analysis in later phases of the project.

2.4 Comparison of Existing Techniques

Approach	Typical Technique	Strength	Limitation
Rule / keyword based	Manual heuristics, blacklists	Simple, fast, explainable	Easy to bypass, poor generalisation
Classical ML	TF-IDF + KNN / LR / RF	Fast to train, interpretable	Ignores word order and context

Approach	Typical Technique	Strength	Limitation
Sequence deep learning	LSTM / GRU with embeddings	Captures word order	Needs more data and compute
Transformer-based	Fine-tuned BERT-family models	Strong contextual understanding	High computational cost
Metadata-augmented	Text + source/sentiment features	Uses signals beyond wording	Depends on quality of metadata

2.5 Research Gap

Across the approaches summarised above, a recurring gap is the trade-off between accuracy and computational cost: the methods that generalise best to unseen, real-world articles are also the most expensive to train and deploy. This project addresses a smaller, more practical slice of the problem — establishing a clean, well-documented preprocessing pipeline — which is the stage on which the accuracy of any downstream classical or deep model ultimately depends.

3. Dataset Description

3.1 Dataset Source

The project uses a locally provided CSV file, GlobalFakeNews_Research2026_v1.csv, loaded directly from disk in the notebook rather than fetched from an online repository such as Kaggle. The file was supplied for this project and is treated here as the dataset of record; no external source could be verified for it, and it is documented as such rather than attributed to a source it did not come from.

3.2 Dataset Size and Structure

The dataset contains 1,200 rows and 14 columns, confirmed using data.shape and data.info() in the notebook. Table 3.1 lists each column along with its data type and a short description.

Column	Data Type	Description
id	int64	Unique numeric identifier for each article
title	object (str)	Headline of the news article
full_text	object (str)	Full body text of the article
source_name	object (str)	Name of the reporting outlet (e.g., BBC, Reuters, CNN)
publish_date	object (str)	Date the article was published
category	object (str)	News category (Politics, Sports, Health, etc.)
country	object (str)	Country associated with the article
label	int64	Target variable: 0 = Real, 1 = Fake
sentiment_score	float64	Sentiment polarity score of the article text
clickbait_score	float64	Score indicating sensational / clickbait phrasing
emotional_word_ratio	float64	Proportion of emotionally charged words
readability_score	float64	Readability estimate of the article text
source_trust_score	float64	Assigned credibility score for the reporting source
text_embedding	object (str)	Pre-computed numeric embedding vector stored as text

3.3 Target Variable and Class Distribution

The target variable is label, encoded as 0 for Real and 1 for Fake. The dataset is perfectly balanced, with 600 articles in each class, as confirmed by data['label'].value_counts() in the notebook. Figure 3.1 shows this distribution.

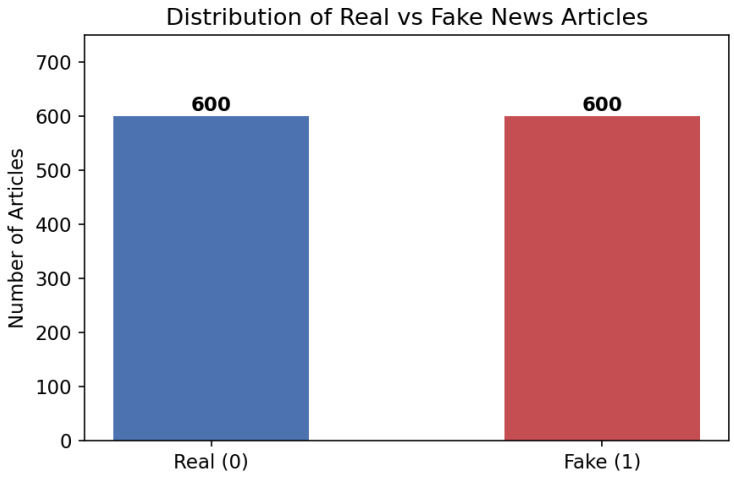


Figure 3.1 Distribution of Real vs Fake News Articles

3.4 Category and Country Distribution

The category column spans seven news categories. International, Entertainment and Sports are the most frequent, while Politics is the least frequent category, as shown in Figure 3.2. Articles are also associated with seven countries, with Australia and India contributing the largest shares, shown in Figure 3.3.

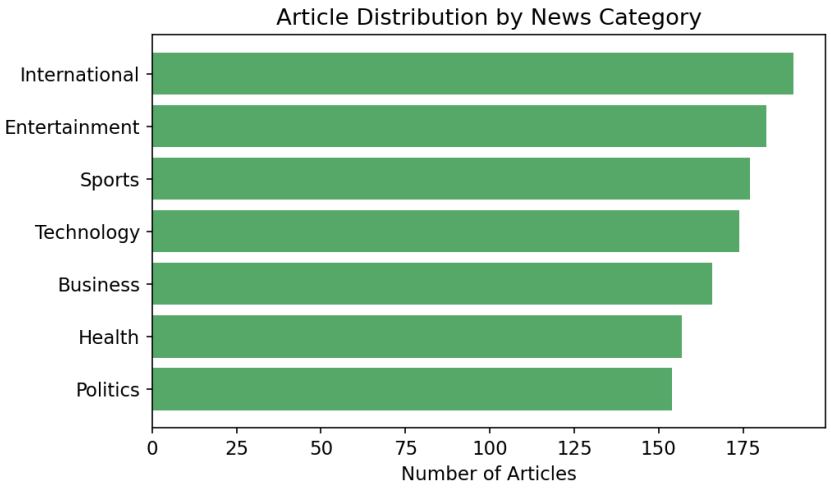


Figure 3.2 Article Distribution by News Category

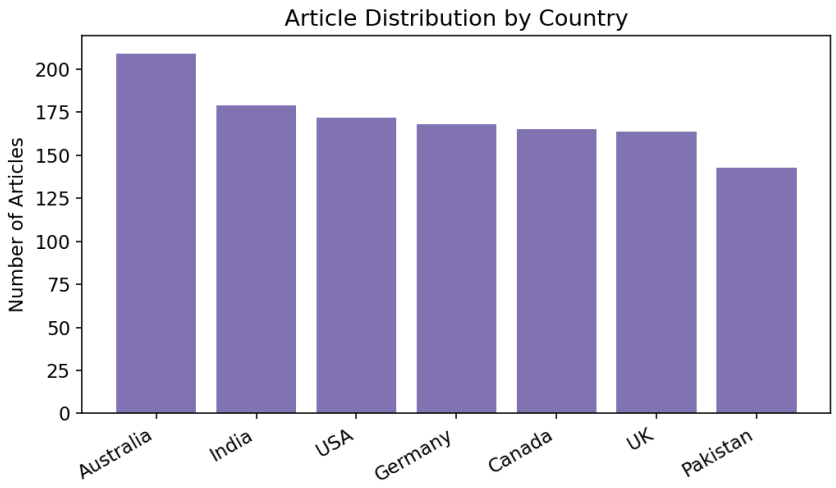


Figure 3.3 Article Distribution by Country

3.5 Auxiliary Numeric Features

In addition to the text fields, the dataset provides five pre-computed numeric indicators for every article: sentiment_score, clickbait_score, emotional_word_ratio, readability_score and source_trust_score. Table 3.2 summarises their range across the dataset. These fields were not used anywhere in the current notebook, but are available for later stages of the project, for instance to combine text-based and metadata-based features.

Feature	Minimum	Mean	Maximum
sentiment_score	-0.5695	0.3538	1.3106
clickbait_score	0.0003	0.4773	0.9996
emotional_word_ratio	0.0507	0.2859	0.5997
readability_score	30.32	55.10	74.98
source_trust_score	0.10	0.55	0.98

3.6 Missing Values and Duplicate Records

A completeness check using data.isnull().sum() confirmed zero missing values across all 14 columns, and data.duplicated().sum() confirmed zero duplicate rows in the dataset. Table 3.3 reproduces this check for the columns most relevant to the text-processing pipeline.

Column	Missing Values
full_text	0
title	0
label	0
source_name	0
category	0

Column	Missing Values
country	0
Total duplicate rows	0

3.7 Observations

- The dataset is perfectly balanced between the two classes (600/600), which removes the need for class-imbalance handling such as oversampling or class weighting in the modelling phase.
- No missing values or duplicate rows were found, so the cleaning stage implemented in this project did not need to perform imputation or de-duplication.
- A closer look at the full_text column shows that many articles are built from a small number of template sentences repeated several times (for example, the sentence “This article provides detailed analysis based on verified sources and official statements.” appears repeated within a single record). This is consistent with the dataset being a synthetically constructed benchmark rather than a corpus of naturally scraped news articles, and it directly explains why the word_count feature derived in Chapter 4 clusters around only two fixed values (35 and 45 words) instead of following a natural, continuously varying distribution.
- The presence of pre-computed columns such as text_embedding and the four auxiliary scoring fields suggests the dataset was designed to support more than plain text classification, and could be combined with the cleaned text in a future extension of this project.

4. Methodology

This chapter documents the preprocessing pipeline exactly as implemented in the notebook, in the order the cells were executed. Figure 4.1 gives an overview of the full sequence before each step is described individually.

Figure 4.1 Text Preprocessing Pipeline Implemented in the Notebook

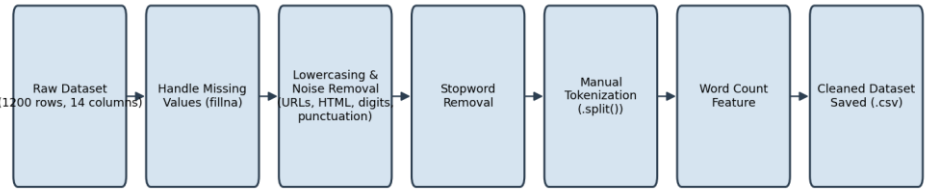


Figure 4.1 Text Preprocessing Pipeline Implemented in the Notebook

4.1 Library Imports

The notebook imports pandas and numpy for data handling, re and string for text manipulation, and nltk for stopword removal. The stopwords corpus is downloaded once using nltk.download('stopwords') before it is used. No machine learning library (such as scikit-learn) or plotting library is imported at this stage, since the notebook has not yet reached the feature engineering or modelling steps.

4.2 Data Loading

The dataset is read directly from a local file path using pd.read_csv(), and a confirmation message is printed once loading succeeds. The dataframe is then inspected using data.head(), data.info(), data.shape and data.columns to verify that all 14 expected columns were read correctly and that the data types match expectations (see Chapter 3, Table 3.1).

4.3 Data Quality Checks

Two checks are performed before any cleaning takes place: a null-value count using data.isnull().sum(), and a duplicate-row count using data.duplicated().sum(). Both checks returned zero, as reported in Section 3.6, which meant the pipeline did not need to include any imputation or de-duplication logic.

4.4 Handling Missing Values

Although no missing values were found, the notebook still applies data['full_text'] = data['full_text'].fillna("") as a defensive step before the cleaning function is applied, so that a null value in this column would not raise an error during string processing rather than silently propagate into the cleaned dataset.

4.5 Text Cleaning Function

A clean_text() function is defined to normalise the article text. It was written in two passes within the notebook: an initial version that removed punctuation and numbers together with a single non-alphabetic regular expression, and a refined version used for the final output that separates these operations and adds whitespace normalisation. The final version performs the following steps, in order:

- Convert the input to a string and lowercase it.
- Remove URLs using the pattern http\S+.
- Remove HTML tags using the pattern <.*?>.
- Remove digits using the pattern \d+.
- Remove punctuation using the pattern [^\w\s].
- Collapse repeated whitespace into a single space and strip leading/trailing spaces.
- Split the text into words and remove English stopwords using the NLTK stopwords corpus.
- Rejoin the remaining words into a single cleaned string.

The cleaned output is stored in a new column, clean_text, alongside the original full_text column so that the effect of cleaning could be inspected directly, which the notebook does using data[['full_text','clean_text']].head().

4.6 Tokenisation

Tokenisation is performed manually rather than through a dedicated NLTK tokeniser: data['clean_text'].apply(lambda x: x.split()) splits each cleaned string on whitespace and stores the resulting list of tokens in a tokens column. This is a simple, whitespace-based approach, which is appropriate here since punctuation has already been stripped out during cleaning.

4.7 Word Count Feature

A word_count column is derived by applying len() to each row's token list. Figure 4.2 shows the distribution of this feature across the dataset. As noted in Chapter 3, the distribution takes only two values (35 and 45 words), which follows directly from the templated, repeated-sentence structure of the underlying article text rather than from any error in the counting logic.

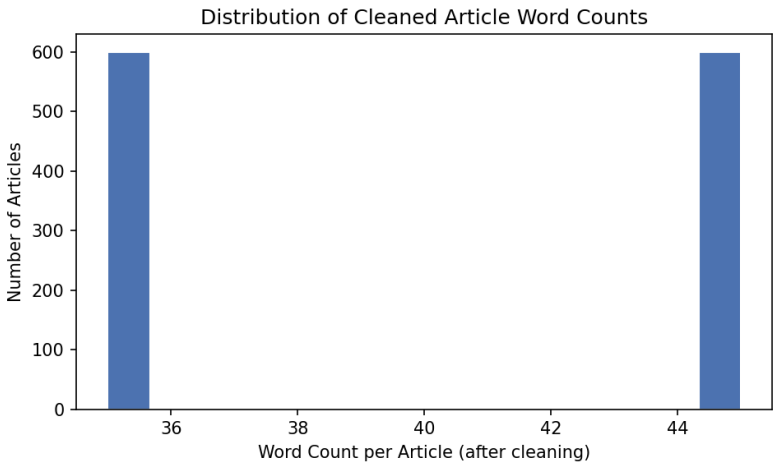


Figure 4.2 Distribution of Cleaned Article Word Counts

4.8 Saving the Cleaned Dataset

Once cleaning, tokenisation and word-count extraction are complete, the notebook saves the resulting dataframe to Cleaned_Fake_News.csv using data.to_csv(), and prints the final shape of the dataframe to confirm that the new columns (clean_text, tokens, word_count) were added successfully, bringing the column count from 14 to 17.

4.9 Feature Engineering and Model Pipeline (Planned)

The steps described above complete the data-preparation stage of the project. TF-IDF or Bag-of-Words vectorisation of the clean_text column, the train-test split, and the construction of a scikit-learn training pipeline have not yet been implemented in the notebook submitted alongside this report. They are described conceptually in Chapters 5 and 6 as the next stage of work, and no code for them currently exists, so no results from that stage are reported here.

Step	Operation	Status
1	Load dataset (pd.read_csv)	Implemented
2	Exploratory checks (shape, info, columns, nulls, duplicates)	Implemented
3	Handle missing values (fillna)	Implemented
4	Lowercase, remove URLs/HTML/digits/punctuation	Implemented
5	Stopword removal	Implemented
6	Tokenisation (whitespace split)	Implemented
7	Word count feature	Implemented
8	Save cleaned dataset to CSV	Implemented
9	TF-IDF / Bag-of-Words vectorisation	Planned — not yet implemented
10	Train-test split	Planned — not yet implemented
11	Model training (KNN, LR, RF, NN)	Planned — not yet implemented

5. Planned Model Development (Future Work)

No classification model has been trained in the notebook accompanying this report; only the text-preprocessing stage described in Chapter 4 has been implemented. This chapter is included to document the modelling approach that was planned at the outset of the project, so that the report can be extended directly once training code is added, without needing to rewrite this section. No hyperparameters or results are reported here, since none currently exist.

5.1 K-Nearest Neighbours (Planned)

KNN is a non-parametric classifier that assigns a label to a new article based on the majority label among its k closest neighbours in the vectorised feature space, typically measured using cosine or Euclidean distance. It was selected as one of the candidate models for its simplicity and its usefulness as a baseline, though it is expected to scale poorly once the TF-IDF feature space becomes large, since distance calculations must be repeated against the full training set for every prediction.

5.2 Logistic Regression (Planned)

Logistic Regression models the probability of an article being fake as a function of its input features through a sigmoid activation applied to a weighted sum of those features. It is a common baseline for text classification tasks because it trains quickly, performs well on high-dimensional sparse TF-IDF vectors, and produces coefficients that are relatively easy to interpret in terms of which words push a prediction toward Real or Fake.

5.3 Random Forest (Planned)

Random Forest builds an ensemble of decision trees, each trained on a bootstrapped sample of the data with a random subset of features considered at each split, and aggregates their predictions by majority vote. It was selected as a candidate model because it can capture non-linear interactions between features that Logistic Regression cannot, at the cost of being less directly interpretable and typically slower to train on high-dimensional text features.

5.4 Neural Network (Planned)

A feed-forward neural network, taking the TF-IDF (or embedding) vector of an article as input and passing it through one or more dense hidden layers before a final sigmoid output layer, is planned as the fourth candidate model. It is expected to require more training data and computation than the other three models, but has the potential to capture more complex feature interactions if the dataset proves large enough to support it.

5.5 Summary

All four models are intended to be trained on the same TF-IDF representation of the clean_text column, using an identical train-test split, so that their performance can be compared fairly once Chapter 4's pipeline is extended with a vectorisation and modelling stage.

6. Model Evaluation (Planned)

Since no models have been trained yet, this chapter explains the evaluation methodology that is intended to be applied once training is complete, rather than presenting results. No accuracy, precision, recall or F1 figures are reported in this report, in line with the requirement that the report should not contain invented results.

6.1 Evaluation Metrics

- Accuracy — the proportion of articles correctly classified as Real or Fake out of all articles evaluated.
- Precision — out of all articles predicted as Fake, the proportion that are actually Fake, which matters when false alarms are costly.
- Recall — out of all articles that are actually Fake, the proportion correctly identified, which matters when missing fake articles is costly.
- F1-score — the harmonic mean of precision and recall, useful when a single balanced metric is needed.
- Confusion Matrix — a 2x2 table of true positives, true negatives, false positives and false negatives, which will make it possible to see whether a model is biased toward one class.

6.2 Planned Comparison Methodology

Once training data for the four candidate models becomes available, the plan is to evaluate each model on the same held-out test split, using scikit-learn's classification_report and confusion_matrix utilities, and to summarise the four models side by side in a single comparison table (accuracy, precision, recall and F1 columns) so that the trade-offs between the simpler models (KNN, Logistic Regression) and the more complex ones (Random Forest, Neural Network) can be discussed directly.

6.3 Note on Reporting

This report intentionally does not fabricate any confusion matrix values, accuracy percentages or comparison charts, since doing so would misrepresent work that has not yet been carried out. Chapter 7 discusses the results that are actually available at this stage of the project — the outcome of the preprocessing pipeline — rather than model performance.

7. Results and Discussion

This chapter discusses the outcomes that are actually available from the work completed, namely the effect of the preprocessing pipeline on the dataset, and the practical limitations observed while carrying it out.

7.1 Outcome of the Preprocessing Stage

Running the cleaning function across all 1,200 articles produced a clean_text column free of URLs, HTML artefacts, digits and punctuation, and with common English stopwords removed. Comparing full_text and clean_text side by side, as done in the notebook, shows the cleaning function behaving as intended — for example, connecting words such as “this”, “on” and “and” are consistently removed while content-bearing words such as “analysis”, “verified” and “sources” are retained.

7.2 Effect on Word Count

The word_count feature derived after cleaning takes only two values across the entire dataset (35 and 45 tokens), which is a direct consequence of the underlying articles being built from a small number of repeated template sentences rather than naturally varying prose. This is a useful early signal: it indicates that, in its current form, the full_text field alone may not provide enough lexical variety for the four planned models to learn a meaningful distinction between Real and Fake articles, and that the auxiliary numeric columns (sentiment_score, clickbait_score, emotional_word_ratio, readability_score, source_trust_score) described in Chapter 3 may need to be incorporated as additional features once modelling begins.

7.3 Limitations Observed

- The dataset appears to be synthetically generated for coursework purposes rather than scraped from real news sources, which limits how far conclusions from this project would generalise to real-world fake news detection.
- No feature vectorisation or model training has been implemented yet, so no classification accuracy can currently be reported.
- The text cleaning function was rewritten once during development (an initial version and a refined final version both appear in the notebook); only the final version's output is used going forward, and the earlier version is documented in Chapter 4 for completeness rather than as a separate result.

7.4 Practical Usability at This Stage

In its current state, the notebook is best understood as a working, tested data-preparation module rather than a deployable detector. It can reliably take the raw CSV file and produce a cleaned, tokenised dataset ready for vectorisation, which is a necessary and often time-consuming step in any text-classification project, but it does not yet perform classification itself.

8 Conclusion

8.1 Summary of Work Done

This project set out to build a system for classifying news articles as Real or Fake. The work completed so far covers the data-preparation stage of that goal: the 1,200-record GlobalFakeNews_Research2026_v1.csv dataset was loaded and examined, found to be complete (no missing values) and free of duplicate rows, and a text-cleaning pipeline was implemented and applied to normalise the article text, remove stopwords, tokenise the result and compute a word-count feature. The cleaned dataset was exported for future use.

8.2 Limitations

The project has not yet reached the modelling stage: TF-IDF/Bag-of-Words vectorisation, the train-test split, and the training and evaluation of the four planned classifiers (KNN, Logistic Regression, Random Forest and Neural Network) remain to be implemented. The dataset itself also appears to be synthetically constructed with repeated template sentences, which may limit the lexical diversity available to any model trained on the full_text field alone.

8.3 Future Work

- Implement TF-IDF (or Bag-of-Words) vectorisation on the clean_text column.
- Perform a train-test split and train the four planned models.
- Evaluate each model using accuracy, precision, recall, F1-score and a confusion matrix, and compare them in a single summary table.
- Investigate combining the auxiliary numeric columns (sentiment_score, clickbait_score, emotional_word_ratio, readability_score, source_trust_score) with the text-based features, given the limited lexical variety observed in Chapter 7.
- If possible, validate the pipeline on a real-world news dataset in addition to the current synthetic one, to test how well the approach generalises.

Overall, the project has established a working and well-documented foundation for the fake news detection task. The next phase of implementation — vectorisation, model training and evaluation — can be added directly on top of the cleaned dataset produced by the current notebook.

References

[1] W. McKinney, “pandas: a Foundational Python Library for Data Analysis and Statistics,” pandas Development Team, 2024. [Online]. Available: https://pandas.pydata.org/docs/

[2] C. R. Harris et al., “Array Programming with NumPy,” NumPy Documentation. [Online]. Available: https://numpy.org/doc/

[3] S. Bird, E. Klein, and E. Loper, “Natural Language Processing with Python,” NLTK Project Documentation. [Online]. Available: https://www.nltk.org/

[4] Python Software Foundation, “re — Regular expression operations,” Python 3 Documentation. [Online]. Available: https://docs.python.org/3/library/re.html

[5] Project Jupyter, “Jupyter Notebook Documentation.” [Online]. Available: https://jupyter.org/documentation

[6] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, 2011. Referenced for the planned modelling phase described in Chapters 5 and 6; not yet used in the current implementation. [Online]. Available: https://scikit-learn.org/stable/documentation.html

[7] GlobalFakeNews_Research2026_v1.csv — dataset file supplied for this project (local file; source institution/repository not independently verifiable).

Appendix — Selected Code Snippets

The snippets below are reproduced from the submitted notebook and correspond to the pipeline described in Chapter 4. Exploratory print statements have been omitted for brevity, only the functional code is shown.

A.1 Imports and Dataset Loading

```
import pandas as pd import numpy as np import re import string import nltk from nltk.corpus import stopwords nltk.download('stopwords')  data = pd.read_csv(r"GlobalFakeNews_Research2026_v1.csv")
```

A.2 Missing Value Handling

```
data["full_text"] = data["full_text"].fillna("")
```

A.3 Text Cleaning Function (final version)

```
stop_words = set(stopwords.words("english"))  def clean_text(text):     text = str(text)     text = text.lower()     text = re.sub(r'http\S+', '', text)     text = re.sub(r'<.*?>', '', text)     text = re.sub(r'\d+', '', text)     text = re.sub(r'[^\w\s]', '', text)     text = re.sub(r'\s+', ' ', text).strip()     words = text.split()     words = [word for word in words if word not in stop_words]     return " ".join(words)  data["clean_text"] = data["full_text"].apply(clean_text)
```

A.4 Tokenisation and Word Count

```
data["tokens"] = data["clean_text"].apply(lambda x: x.split()) data["word_count"] = data["tokens"].apply(len)
```

A.5 Saving the Cleaned Dataset

```
data.to_csv("Cleaned_Fake_News.csv", index=False)
```